

LiU-ITN-TEK-A--08/081--SE

Dynamic Controllable Mesh Deformation in Interactive Environments

Johannes Rajala

2008-06-11



Linköpings universitet
TEKNISKA HÖGSKOLAN

LiU-ITN-TEK-A--08/081--SE

Dynamic Controllable Mesh Deformation in Interactive Environments

Examensarbete utfört i medieteknik
vid Tekniska Högskolan vid
Linköpings universitet

Johannes Rajala

Handledare Tatu Blomberg
Handledare Panu Liukkonen
Examinator Matt Cooper

Norrköping 2008-06-11

Upphovsrätt

Detta dokument hålls tillgängligt på Internet – eller dess framtida ersättare – under en längre tid från publiceringsdatum under förutsättning att inga extraordinära omständigheter uppstår.

Tillgång till dokumentet innebär tillstånd för var och en att läsa, ladda ner, skriva ut enstaka kopior för enskilt bruk och att använda det oförändrat för ickekommersiell forskning och för undervisning. Överföring av upphovsrätten vid en senare tidpunkt kan inte upphäva detta tillstånd. All annan användning av dokumentet kräver upphovsmannens medgivande. För att garantera äktheten, säkerheten och tillgängligheten finns det lösningar av teknisk och administrativ art.

Upphovsmannens ideella rätt innefattar rätt att bli nämnd som upphovsman i den omfattning som god sed kräver vid användning av dokumentet på ovan beskrivna sätt samt skydd mot att dokumentet ändras eller presenteras i sådan form eller i sådant sammanhang som är kränkande för upphovsmannens litterära eller konstnärliga anseende eller egenart.

För ytterligare information om Linköping University Electronic Press se förlagets hemsida <http://www.ep.liu.se/>

Copyright

The publishers will keep this document online on the Internet - or its possible replacement - for a considerable time from the date of publication barring exceptional circumstances.

The online availability of the document implies a permanent permission for anyone to read, to download, to print out single copies for your own use and to use it unchanged for any non-commercial research and educational purpose. Subsequent transfers of copyright cannot revoke this permission. All other uses of the document are conditional on the consent of the copyright owner. The publisher has taken technical and administrative measures to assure authenticity, security and accessibility.

According to intellectual property law the author has the right to be mentioned when his/her work is accessed as described above and to be protected against infringement.

For additional information about the Linköping University Electronic Press and its procedures for publication and for assurance of document integrity, please refer to its WWW home page: <http://www.ep.liu.se/>

Abstract

The purpose of this project was to research and develop methods for managing the deformation of complex geometric meshes with specific reference to interactive computer games. Bugbear Entertainment has a proven record of accomplishment in the development of car racing games. The creation and management of deformable meshes in real time is a vital component in such games, since it allows the player to visually experience the effect of a real-life car accident. This thesis is meant for people having basic understanding of computer graphics simulation in general, as well as a basic mathematical background for the matrix algebra.

Acknowledgments

I would like to thank the following people who directly or indirectly contributed to the success of my thesis:

Henna and Elias Rajala, my parents, Matthias Müller, Sasan Gooran, Roger Johannesson, Quan Ho, Mattias Unger, my supervisors, and everyone at Bugbear Entertainment.

Contents

1	Introduction	1
1.1	Introduction to Deformation	1
1.2	Collision and Deformation in Car Games	2
2	Theoretical Basis	5
2.1	Studies about Deformation	5
2.1.1	Mass-Spring Method	6
2.1.2	Free Form Deformation	9
2.1.3	Finite Element Analysis	11
2.1.4	Mesh-Free Methods	13
2.2	The Method of Choice	16
3	Implementation	19
3.1	Deformation Framework	19
3.1.1	Data Structures	20
3.1.2	General Operation Flow	20
3.2	Additional stiffness	25
3.3	Deformation Editor	26
3.4	Integration with the Game Engine	29
3.5	Plasticity extension	30
3.6	Transforming normals	32
4	Conclusions	33
4.1	Results	33
4.2	Future Work	34
4.3	Summary	34
	Bibliography	35
A	Deformation Methods in Use	37

Chapter 1

Introduction

Real time computer simulation of dynamic collision and deformation has increased its popularity as a research subject. Techniques for deformation have been under research for quite some time, but the required mathematics have been computationally reasonable only for off-line solutions. Some examples of which can be seen in the movie industry and other off-line animation. Recently both the console and PC hardware have gained enough processing power to allow deformation simulation to be approximated in real time. In highly interactive applications, the restrictions are very strict when considering the computation time but, on the other hand, more relaxed with respect to accuracy. This notion has been used in many approaches attempting simulation at interactive frame rates. In this thesis, different approaches are evaluated for the purpose of finding a solution for deforming cars and other dynamic objects, constructed from different materials, in car racing games. This introductory chapter briefly explains deformation as a physical phenomenon as well as giving examples how it is used in car games.

1.1 Introduction to Deformation

Object's deformation is a result of an event where physical force has enough influence on the object. Hooke's law [7] describes this with a linear relationship between stress (force) and strain (deformation). In this thesis, the type of deformation in focus is the one resulting after a physical collision. Therefore, the following definitions are given.

Definition 1.1 (Collision [3]) *A collision is an isolated event in which two or more bodies (the colliding bodies) exert relatively strong forces on each other for a relatively short time.*

Definition 1.2 (Deformation [19]) *A deformation is an alteration of shape, as by pressure or stress.*

The research about deformation has been done in the field of material research, one of the first researchers being physicist Robert Hooke in the 17th century. He examined materials having linear relationship for the stress-strain relationship. Mostly Hooke's law is referred when springs are involved in simulation, but the theories are applicable to other forms of non-rigid bodies as well. The theory has been extended to also support materials with plasticity. The stress-strain curve in figure 1.1 gives a good description of different phases of deformation. The curve varies among the materials, but the key features can be found for any material.

The figure 1.1 shows that materials have elastic deformation linear to the stress force applied. The elastic strain will restore its original form after the stress is released. When the stress applied reaches a certain level, some elastic materials will experience fracture and the plastic region does not exist or is very small. For mostly ductile materials, the elastic region can be very small and, if the stress is applied more, those experience plastic deformation. This means that they will not restore their original form when the stress is released. Even the ductile materials experience fracture when the limit is reached. Using the features of this curve, many kinds of materials can be simulated from rubber to metals and clay.

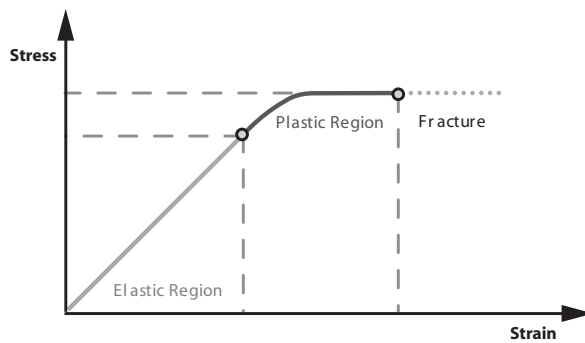


Figure 1.1. Material Stress-Strain Curve shows the linear elastic region followed by plastic region and fracture.

1.2 Collision and Deformation in Car Games

As computer- and console gaming has increased in popularity and become more accepted as a mainstream entertainment, the demands for realism and more convincing effects have increased. Depending on the type of game, the level of interactive action might vary, but in general, the nature of most games is that the action and its response can be performed and experienced in real time. Car games are

one of the most demanding game types in terms of real time interaction. Within car games, the user can easily notice lag in vehicle control or see if the screen skips frames during a high-speed race. For this kind of game, the real time destruction has specific needs in terms of performance and realism.

The first game allowing some form of car deformation and part disassembly was *Destruction Derby* in 1995. The deformations were pre-calculated and then performed according to collision. Other car games worth mentioning because of their deformation systems are *Carmageddon*, *Colin McRae Dirt*, *Toca Race Driver*, *Flatout Ultimate Carnage*, and *Burnout Paradise*. In addition, other game types, such as first-person shooters (FPS), have had their coverage in the real time model deformation as well but those are not covered here, as the problems there are different. Namely, the restriction of field of view and the speed of the objects to be deformed are quite different in FPS games when compared to car games, which gives different possibilities for the solution of the problem.

For detaching parts from the car, most, if not all, of the above games, use car models that consist of several interconnected meshes. When the connections receive enough stress from the physics engine, the different parts will be detached from the main body. In addition to this, the current techniques use some kind of dynamic deformation technique. These may, in their simplest form, be a linear transform from an original non-deformed mesh to a fully crashed one. The latter mesh can either be manually generated or created using some tools provided in the 3D-modelling packages. While this provides visually pleasing results, the deformations are restricted by the previously constructed crash mesh. In addition, this does not allow more extreme dynamic deformations such as bending or twisting.

The increasing customer demand for realism and more impressive effects for real time simulations has driven the research to find solutions to the mentioned problems. Some of these extend the existing off-line solutions with improved algorithms, simplifications and pre-processing, and some are approaching the subject from other non-conventional perspectives. This thesis examines some of the recent techniques at a general level evaluating their benefits and possible problems. After the methods have been discussed, one is chosen for implementation and described in more detail.

Chapter 2

Theoretical Basis

This section describes the theoretical basis for the work presented in the implementation part, and an overview of the available techniques that were required by the nature of the thesis. As there are various methods currently in use for solving the deformation simulation problem, the current methods in use had to be evaluated in order to find the most suitable one for the requirements of the real time implementation and the scope of the thesis.

When considering deformation as a phenomenon, Hooke's law of elasticity [7] cannot be neglected as it was the first approximation defining the linear relationship between the amount of deformation and the force causing it. The law holds true with linear-elastic materials but has to be extended to support plasticity; the condition in which some materials hold the deformation without returning to their original form after the stress force has been released. The previous figure 1.1 illustrated the point where the linear relationship ends and the plastic deformation begins. This kind of behaviour is also expected with the materials involved with car crashes, which can be considered almost completely non-elastic with higher speeds. To obtain results that follows this principle there are some options in the methods currently in use.

2.1 Studies about Deformation

As mentioned before, the subject of deformation has increased popularity within the gaming industry, especially as the computational capabilities have increased to the level that many simulations that were not possible to calculate in real time now are. However, the calculation of all the physical properties of a material at the molecular level still remains insolvable, and at least some approximations and data pre-processing have to be made in order to run simulations that appear dynamic in real time.

There exist a few studies reviewing various methods used in computer graphics for deformations. Two of the more extensive ones are *Physically Based Deformable Models in Computer Graphics*[13] and *A Survey of Computer-Based Deformable Models*[12]. The latter article divides the techniques into three categories: non-physical-, physical-, and approximate physical- techniques. It also covers fewer techniques but in more detail. The division into these three categories may not be the best approach as, for example, the finite element methods are labelled under *Physical techniques* because they are derived from continuum models. However, the basic idea of FEM is to represent materials in a discrete manner, thus making it more an approximate technique. The former article is focused on physically-based techniques, and similarly divides them to three categories: Lagrangian Mesh Based, Lagrangian Mesh-Free, and Mesh-free. Appendix B contains a table that was constructed to summarize the properties of the methods presented in the first [13] article.

Some of these are still computationally too heavy to be considered for real-time applications. In most real-time applications the speed has to be favoured over accuracy. There is also gap between what is visually pleasing and what is physically correct. The rest this section introduces in more detail some of the methods mentioned in the above-mentioned reviews.

2.1.1 Mass-Spring Method

Mass-spring based methods are widely used for various things in the gaming industry, for example cloth- and rigid body simulation. The key idea behind the model is to have mass particles interconnected with springs. These springs can have different sets of rules or constraints. Basic constraints are length and angle, but can contain other material properties as well. The simulation uses a set of ordinary differential equations (ODE) to hold the constraints as close to the set values as possible, while forces affect and move the mass particles. Below the figure 2.1 shows a mass spring structure that could be used for cloth simulation and another with infinitely stiff springs that could be used for rigid body simulation.

There are two basic steps in the mass spring simulation. The first is to apply the physical forces to particles and move them correspondingly. The second step is to apply the constraints given for the system, or individual particles. For the first step, determining positions for the particles, an explicit forward Euler integration shown in equations 2.1 and 2.2 can be used before applying constraints.

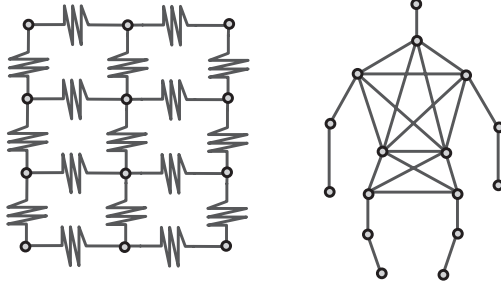


Figure 2.1. A mass spring cloth and a rigid body using stiff springs

$$\begin{aligned} v'_i &= v_i + a \cdot \Delta t \\ &= v_i + \frac{\Delta t F_i}{m_i} \end{aligned} \quad (2.1)$$

$$x'_i = x_i + \Delta t v'_i \quad (2.2)$$

A developer and researcher in gaming industry, Thomas Jacobsen, suggests[8] another, velocity-less integration scheme to improve stability of integration. The Verlet integration was developed by French physicist Verlet in 1967 [20], and is more commonly used in molecular dynamics simulation. It keeps track of each particle's current and previous position, and calculates the next one using those and the acceleration. The equation 2.3 shows how to integrate for the next position using the Verlet integration. In addition to increased stability, the Verlet integrator also has the property of time-reversibility that is useful when implementing collision response system.

$$x'_i = 2x_i - x_i^0 + a \cdot \Delta t^2 \quad (2.3)$$

$$x_i^0 = x_i \quad (2.4)$$

In some cases the velocity is needed for further processing, such as calculating the kinetic energy for energy conservation ($E = K + V$). In these cases one can use the equation 2.5 which, however incorporates an error of the order of Δt^4 . Other option is to use a modified version of the Verlet integrator such as the *Leap-frog* algorithm [9] or the *Velocity Verlet* integrator [4].

$$v = \frac{x'_i - x_i^0}{2\Delta t} \quad (2.5)$$

Mass spring systems rely heavily on constraints, which can describe various properties of the material to be simulated. Even some information about the environment

restrictions can be configured as constraints. A simple collision constraint of a box from $[-1,-1,-1]$ to $[1,1,1]$ can be defined as in the equation 2.6.

$$x_i = \min(\max(x_i, -1), 1) \quad (2.6)$$

A more useful constraint is a distance constraint, which can be set between two connected mass particles. This can be defined as shown in the following. Here the x_1 and x_2 are the mass particles and d is the distance to be enforced.

$$|x_2 - x_1| = d \quad (2.7)$$

The distance constraint is satisfied by moving the end-points either towards or further away from each other, as shown in the following equations 2.8. Here the d is the distance to be enforced, and x_2 and x_1 are the current positions of the particles, half of the needed difference from the desired distance is added to both particle positions.

$$\begin{aligned} b_1 &= x_2 - x_1 \\ b_2 &= \sqrt{(b_1)^2} \\ b_3 &= \frac{b_2 - d}{b_2} \\ x_1 &= x_1 + \frac{1}{2}b_1b_3 \\ x_2 &= x_2 - \frac{1}{2}b_1b_3 \end{aligned} \quad (2.8)$$

In order for the simulation to fulfil both of the constraints they would have to be solved simultaneously by a system of equations. Depending on the required accuracy the solution also can be approximated by iterating both of the constraints for a chosen number of times. In this approach the system eventually converges to the desired solution as well. This method is called relaxation, or Jacobi iteration. This kind of method is especially useful with multiple constraints to be satisfied.

The above is all that is required for a functional mass spring system. The article by Jacobsen [8] in 2001 provides an example of mass-spring related method, with the focus on rigid bodies, and offers detailed information on implementation issues. His method has been used for character simulation in the game *Hitman*.

The mass-spring model, while being easy to construct, has some problems in realistic simulation of various kinds of materials. The constraint properties and damping can be difficult to adjust. In addition the springs may have to have many cross bindings over the model to prevent shearing. Constructing the spring network may become more tedious when the required amount increases and the

computational burden increases simultaneously. There is also a problem when using strong springs to construct stiff objects, as it results in numerical instability with simple integration methods. For a rigid body implementation using Verlet integration, the stability is not an issue.

2.1.2 Free Form Deformation

Free form deformation (FFD) was one of the first techniques that did not perform the deformation directly on the object. When using the FFD method, the object is embedded into a space, which is deformed. For the purposes of FFD, the model is divided into a lattice, each cell having a space warping function first introduced by Barr 1984 [1]. These functions supported scaling, tapering, twisting and bending. When the lattice is deformed, the embedded object will follow. The generalisation of the functions and use of lattices was done by Sederberg et al. [17] in 1986, where they used structural hyper patches for the deformation, see figure 2.2. The methods presented then are still in use, but the technique has been extended to support different types of lattices, animation and direct manipulation. Many modelling packages provide tools for free form deformation, because of its intuitive interface.

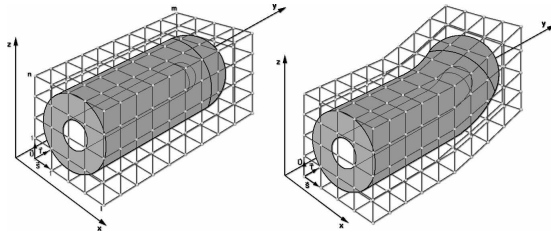


Figure 2.2. Original non-deformed object inside the lattice, and a FFD-deformed object.

Conceptually the deformation is handled by using a set of lattice control points which, when moved, are transformed through a blending phase to final locations. The lattice structure is composed of tricubic hyper patches. For a single FFD rectangular block, each face is a hyper patch. Sederberg states that the points inside an FFD block are given local coordinates according to the patch. After the local coordinates have been initialized, the control points are translated by the user interaction or simulation. Then the deformed position for the control points and points inside the blocks is calculated using so called blending functions defined by the degree and a chosen formulation.

Bezier patches are one option, but for fine detail deformation, the increased number of control points also increases the degree of the curve. Moreover, the control

points would be affected even at a considerable distance from the actual deformation, which is not the desired effect. Therefore, instead of Bezier curves, cubic B-splines (actually B-spline hyper patches) are used, as they allow more local control of the FFD. B-splines are actually a generalisation of Bezier curves. Figure 2.3 shows a curve defined by five control points in two dimensions; the hyper patch is an extension to three dimensions. [5]

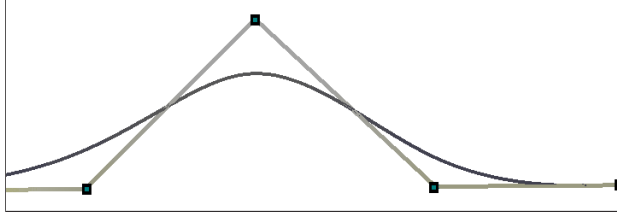


Figure 2.3. A cubic spline curve defined by five control points.

Blending of the particle locations as Sederberg et. al describe, is done with so called *Bernstein blending functions*. For the non-deformed object, a rectilinear coordinate system is assumed. Similarly, for the deformed model a curvilinear coordinate system is used, where the Bernstein blending functions define the axes. Equation 2.9 shows the formulation, where the control points $P(x, y, z)$ are represented in a parametric form, $P(u, v, w)$.

$$P(u, v, w) = \sum_{i=0}^3 \sum_{j=0}^3 \sum_{k=0}^3 p_{ijk} B_i^3(u) B_j^3(v) B_k^3(w) \quad (2.9)$$

This function is used to find new positions for all the points in the model. The polynomials used in the construction of the hyperpatches are not limited to certain degrees. As described before however, if the requirement is to have local deformations as well, the FFD has to be applied piecewise and formulations such as cubic B-splines can be used. It should be noted that the order of the B-spline functions could be freely chosen.

The B-spline curve defined by equation 2.10 has n control points and the curves have order $d \in [2, n + 1]$. The $B_{k,d}$ are the blending functions of degree $d - 1$. P_k are the control points. Each of the $B_{k,d}$ is non-zero only in a relatively small range of t , and so the curve provides local control. Each B is defined recursively. In addition, a relevant aspect of B-splines is the definition of *knot vectors*. Knot vectors are an increasing sequence of values such that a vector is defined as $(t_0, t_1, \dots, t_{n+d})$, where $t_0 \leq t_1 \leq \dots \leq t_{n+d}$. In uniform cubic B-splines the knot vector is of the form $(-3, -2, -1, 0, 1, \dots, n + 1)$.

$$X(t) = \sum_{k=0}^n P_k B_{k,d}(t) \quad (2.10)$$

The blending function, B , for a cubic spline is given by equation 2.11 below.

$$B_{0,4}(t) = \frac{1}{6} \begin{cases} (t+3)^3 & -3 \leq t < -2 \\ -3t^3 - 15t^2 - 21t - 5 & -2 \leq t < -1 \\ 3t^3 + 3t^2 - 3t + 1 & -1 \leq t < 0 \\ (1-t)^3 & 0 \leq t < 1 \end{cases} \quad (2.11)$$

A simple FFD system has been described above and, as mentioned, there have been many extensions to the basic formulation. The basic problem, even when the cubic B-splines do allow local controls, is that a well-formed structure is required for the deformation restricting the models to usually consisting of only uniform solids. The control of material properties is not very intuitive either.

2.1.3 Finite Element Analysis

Finite element analysis was developed for structural dynamics, calculating aeroelastic effects for air-plane wings and, later, for the needs of the space program. In 1956 Turner et.al. discovered a better way to calculate shear fields using simple strain fields, and released their article *Stiffness and deflection analysis of complex structures* [10], which described the finite element analysis basics as still in use [16].

Overall, finite element analysis is a simulation that uses the numerical technique finite element method (FEM) to obtain solutions. The purpose of the analysis is to conserve energy in the system where physical reactions occur. As integrating functions defining the energy for irregularly shaped models are complex to solve, the division to discrete finite elements makes analysis easier. The object in question is divided into an assembly of finite elements.

In their article Turner et.al. describe three types of finite elements: triangular, quadrilateral and rectangular that could be used to solve two-dimensional elasticity. For example, the triangular element consisted of three constant strain components and three rigid body displacements. Using Hooke's law [7] the stress components can be calculated from the nodal displacements. The boundary stress components can be replaced by equivalent nodal forces giving the stiffness matrix.

Figure 2.4 shows the basic element types in one, two and three dimensions, also some examples of linear and higher order shapes are given. For three dimensional simulations the two-dimensional elements can be used in boundary element method (BEM), which operates only on the model boundary. For FEM in three dimensions, a three dimensional element is required. There exist both free and commercial applications that can create the internal structure for a model using

tetrahedrons or other elements. The points in the figure represent nodes. In FEM each of the nodes store nodal variables which are used as parameters for differential equations, which are solved. Nodal values contain information about object material properties.

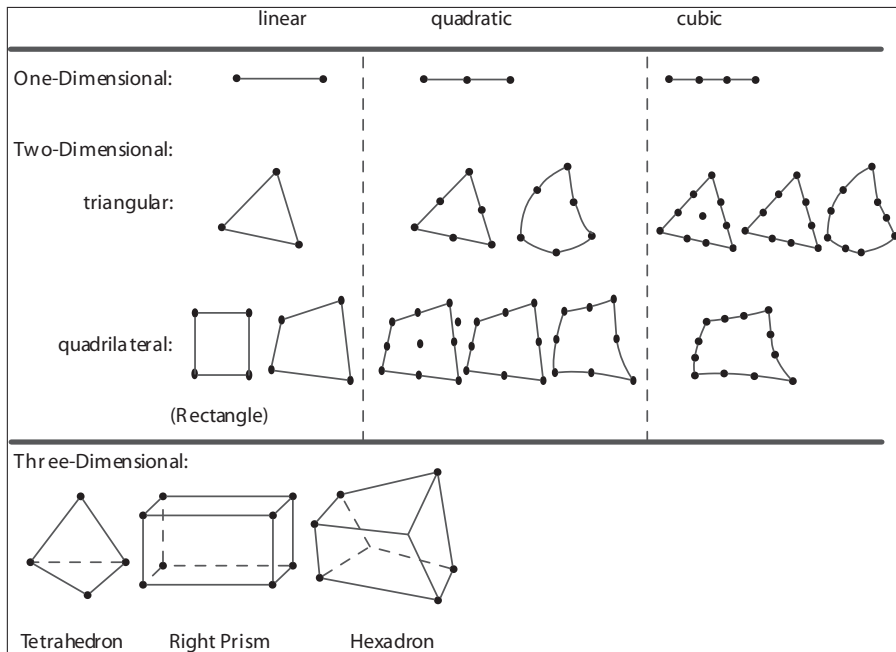


Figure 2.4. Basic Finite Element Types

The most commonly used method to find appropriate finite element approximations is the so-called Ritz-Rayleigh procedure. The procedure does behaviour analysis of each element type in terms of loads and responses, trying to find the minimal solution to the system. The result of the analysis is usually represented by a matrix, for which the size is determined by the type of the element and degrees of freedom of the nodes. For a triangle with nodes of one degree of freedom, the matrix is 3-by-3, but for a triangle with nodes having three degrees of freedom (x, y and x displacement), the matrix is 9-by-9. For each individual element, its matrix components are given by its shape function and the nodal values.

After the matrices are computed for each element those are combined together into one large matrix representing the whole system. The combination takes into account the shared nodes and weights to get a system equation. Any forces applied to individual nodes can be added to this equation before finally solving the system

by using for example Gauss-Jordan elimination.

The FEM is the most realistic method currently in use, but has a problem of computational overhead. When there are many objects composed of dense tetrahedron sets, all requiring a solution of large matrices the overhead quickly increases too much to handle in real-time. Recently, however, there have been advancement in this section with some methods claiming near interactive operation. A company called Pixelux has investigated the method and implemented a solution, based on FEM, named Digital Molecular Matter (DMM). DMM also claims to have a capability for real time fracture simulation of different materials. The method has not been publicly presented in detail but it does rely on some pre-processing of the meshes to create the internal volume information. The DMM is not publicly available at the time of writing, and thus its performance cannot be evaluated in action. Thus, when the need is to have tens or even hundreds of objects deforming at the same time and, in addition, to do other computations as necessary for a gaming environment, there currently has not been a publicly open FEM solution to the problem.

2.1.4 Mesh-Free Methods

The mesh-free methods do not consider the mesh connectivity when calculating physical responses. The mesh is point sampled and particles simulated without connectivity information. The mesh-free methods are related to point-based animation in the sense that they consider the mesh to either consist of points or for it to be sampled to a set of points. The point-based approach gives many advantages as opposed to the mesh-based methods. One of the most important benefits being the ease of computation when doing the physical simulation when there is no inter-connectivity relationships to be considered.

The mesh-free method *Meshless deformations based on shape matching* described by Müller et al. [11], uses the least squares method to match the relative locations of the particles to their corresponding original locations. This kind of matching compensates for the lack of actual connectivity information by the use of information about the original positions. The method aims to keep the proportions intact when the object has been rotated or deformed in the mesh-free simulation phase. The shape matching steps are illustrated in the figure 2.5. The original particle positions are used in calculation of the original transformation, and then the particles can be moved in a physics simulation simulating gravity, collision responses, etc. Finally, these positions are used in the calculating another transformation representing the current state of the positions. The original and the current transformations are used in finding the optimal transformation and thus the goal positions for the particles. This final transformation makes the particles keep correct proportions of their relative positions even after simulation with large time steps, as the shape matching is done towards a stable original state.



Figure 2.5. Shape matching: original configuration, result of particle-based physics simulation, shape matched position correction

The simplicity of the calculations in the shape matching approach allows real time performance for multiple complex objects. The limitations of the method come in the form of flexibility. Using linear functions to match the original shape allows only for simple linear deformations, i.e. scaling, stretching, and shearing. To allow quadratic features, such as bending, the least squares have to be quadratic as well.

The initial mesh is sampled to get a discrete set of control points, this set is defined as x_i^0 . The points are scaled by their masses, which gives the centre-of-mass defined as shown in equation 2.12

$$x_{cm}^0 = \frac{\sum_i^n m_i x_i^0}{\sum_i^n m_i} \quad (2.12)$$

Another similar set is the set of control points, x_i , produced after a simulation step where the particles are moved just by adding effects of the forces such as gravity and/or collision response. This set has a similarly defined x_{cm} , where the positions are picked from the set. This set is an intermediate set that is used in the shape matching method for achieving the final goal positions. The final positions are then found by blending between the goal positions and the actual positions according to the stiffness factor, α , but first a definition is required in order to calculate the matrices needed for shape matching.

As described before, the method presented solves the optimal transformation A in a least-squares sense, thus the following equation needs to be minimised:

$$\sum_i^n m_i (A(x_i^0 - x_{cm}^0) - (x_i - x_{cm}))^2 \quad (2.13)$$

To simplify the formation the following definitions are made in the paper:

$$q_i = x_i^0 - x_{cm}^0 \quad (2.14)$$

$$p_i = x_i' - x_{cm}' \quad (2.15)$$

This gives the new formula for equation 2.13:

$$\sum_i^n m_i (Aq_i - p_i)^2 \quad (2.16)$$

The optimal transformation is found when the derivatives with respect to coefficients of A are set to zero [11]:

$$A = \left(\sum_i^n m_i p_i q_i^T \right) \left(\sum_i^n m_i q_i q_i^T \right)^{-1} = A_{pq} A_{qq} \quad (2.17)$$

For a simulation of a completely rigid body the rotational transform has to be separated from the scaling and stretching part. This is possible with a mathematical matrix factorization operation called *polar decomposition*. There are different ways to perform this calculation in practice, but Müller suggests using a Jacobi method iterating towards the solution. Shoemake et al [18] has also outlined another fast implementation. They also give an extensive introduction to polar decomposition, its problems and examples where it can be used.

$$\begin{aligned} A_{pq} &= RS \\ S &= \sqrt{A_{pq}^T A_{pq}} \\ R &= A_{pq} S^{-1} \end{aligned} \quad (2.18)$$

The equations in 2.18 does not use the matrix A_{qq} at all, because the it is a symmetric matrix containing only scaling. Usually however, the materials to be deformed are not completely rigid by their nature. In such cases, the optimal transformation can be defined as in equation 2.19 below. Here the β is a blending factor, which can be defined by the material properties of the object under simulation.

$$T = \beta A + (1 - \beta) R \quad (2.19)$$

The matrix T can then be used to compute the set of goal positions g_i .

$$g_i = T q_i + x_{cm} \quad (2.20)$$

And the goal positions are then used in equations 2.21 and 2.22 for finding the actual positions for the next timestep. Here the h is timestep, v the velocity and the F_{ext} the external forces having effect on a particle.

$$v_i(t+h) = v_i(t) + \frac{\alpha}{h} (g_i(t) - x_i(t)) + \frac{h F_{ext}}{m_i} \quad (2.21)$$

$$x_i(t+h) = x_i(t) + h v_i(t+h) \quad (2.22)$$

In [11] the authors prove the unconditional stability of the method and that it does not introduce damping in comparison to discrete Euler integration.

Extensions and improvements

In order to use more complex transformations than stretching, scaling and rotation one could increase the computational complexity in the least-squares part. By using a quadratic formulation, the resulting matrices A_{pq} and A_{qq} are sized 9-by-9. This increases the requirements on memory and computation. To overcome some of the limitations in transforms, but still using the linear model, the mesh can be divided into clusters. The clustering increases the possible deformation and allows the usage of linear functions. Müller describes the clustering as one of the extensions to his basic method and also discusses how to implement plastic deformation in addition to purely elastic one. As mentioned before in the beginning of chapter 2, the plastic deformation is a highly desired feature in the simulation of metallic deformation, making the meshless shape matching an attractive topic for research.

The shape matching method by Müller was released in 2004, and has thereafter gained some interesting extensions such as object cutting[6] and also similar implementations allowing fractures and improved smoothness between the cells[15]. The paper by Rivers [15], describes the fracture in more detail and improves on the calculation routines (namely the caching of the input for Jacobi-method used for polar decomposition) and overall stiffness in a clustered system, but the paper does not discuss plasticity or provide means for achieving linear transformation. Some problems still do arise, namely with the interpenetration, and the complexity of the deformation.

2.2 The Method of Choice

Similarities between the methods

Earlier in this chapter the most prominent solutions for real time deformation simulations were reviewed. While many of them have their problems and limitations, they also shared some similarities. All of the methods have a concept of a *control particle*, which is used to store physical data about its neighbourhood. These particles are used when calculating the results of physical simulation. These physical particles use some sort of connectivity structure that is usually of a lower resolution than the actual vertices of the model's mesh to be simulated. Usually, the physics simulation operates on the particles and after the result is found, some physical formulae or constraints are satisfied and particles are translated to their final locations. The real mesh vertices are moved either by a geometrical relationship to the physics particles or as a result of other mathematical operation, for

which the particles have an effect on.

Taking into account the similarities, many of the methods could in theory run in parallel for a specific solution for each special problem case, using the same or similar properties. However, for the requirements presented in this thesis, both in the time and in the scope, only one optimal solution for the case of real-time plastic deformation was chosen. For further testing and method comparisons, the framework is left extensible to support additional, hybrid or improved techniques.

Issues with different methods

The mass-spring solution has its problems in the automatic construction of a spring network that would have enough coverage for complex meshes. In addition, there are issues finding suitable constraints with reasonable variables.

The free-form deformation is generally considered a good option for mesh editing and simulating elastic features of materials. Solving the Bernstein functions is computationally feasible even for real time applications. The main issue with the FFD method is that the generated grid has limitations to its form and many complex objects cannot be defined without specifying several sub-sections throughout the mesh.

The finite element analysis seems to give impressive results and it can be fine-tuned to simulate many detailed physical effects. Still, the computational overhead of the currently existing implementations remains too heavy for it to be realistically considered for a simulation with tens to hundreds of simultaneously dynamically deforming complex objects. The FEM clearly has a promising future when the computation power is sufficient for doing other game mechanics and physics simulation in addition to finite element analysis. When real time implementations become available, the modeling itself has to have a support for giving accurate physical parameters. Even the most simple objects require these for those not to shatter into pieces being too thin or by having a density value too low. Moreover, a pre-processing step for creating the internal geometry is likely to be required, as different materials tend to fracture differently depending on their internal structures.

The shape matching method for meshless simulation introduced by Müller et. al [11] is an interesting system when considering the real-time restrictions and flexibility for simulating different materials and plasticity. The mathematics involved is efficient enough for real-time applications and the system itself is stable for variable time steps. Some problems do arise with object interpenetration since the connection information is not stored. However, when the cell cluster is connected so that the empty interiors are filled as well, the inter-penetration problem should not occur. When using linear transformations the deformation movement

is also limited. By the use of a cell cluster, it is also possible to simulate quadratic effects, such as bending, by increasing the cluster density. The usage of linear transformations is much more efficient in its memory footprint in comparison to using quadratic ones. In addition, the computational efficiency is considerable, especially in the plasticity case, where both the polar decomposition and the inverse have to be done for 9-by-9 matrices.

The Selection

Given the issues and similarities with each of the methods, when selecting an appropriate method for a such a demanding real time application as a computer game engine, the highest importance has to be given to the methods' real-time capabilities and stability. With the information available and presented above for each of the methods, and given that the time resources for implementing a fully-fledged solution for each of them are not sufficient, there is no other reasonable solution than the meshless shape matching method presented by Müller et al. Therefore, the implementation section of the thesis will be covering most of the details and implementation specific issues around the creation of a deformation framework based on meshless shape matching.

Rotation Inclusion Improvement

While running tests on a system similar to the one described in Müller's paper an improvement for the method was tested with positive results. The section describing the plasticity extension in the original paper shows how the final plastic deformation contains only the linear part of the transformation. This is sufficient when there are only a few clusters but it already presents a limitation to the plasticity. With many interconnected cells, the linear-only deformations become unstable in larger deformation. To improve this problem the method was modified to account for the rotation into the plastic matrix as well. This allows the simulation of some effects similar to the quadratic simulation with only linear complexity, such as bending, although only in a more discrete manner. In the chapter 3.4 this is visualised: the figure 3.9 shows a case of stacked cubes with a plastic deformation in linear case and the figure 3.10 the same in a rotation-included case. The inclusion of the rotation requires the update of static matrices, A_{qq} , as in the linear case, but also the update of the original centres-of-masses.

Chapter 3

Implementation

The purpose of the thesis was to find a suitable method for performing dynamic mesh deformation in real time. In the research chapter of the thesis the most used theories were studied and evaluated on a theoretical level. One of the methods was chosen for the final implementation: *Meshless deformations based on shape matching* [11]. Where the previous chapter presented the theory behind the method, this chapter discusses the method at the implementation level. The implementation was made in the form of a deformation framework adaptable for use in external editor as well as in an actual game engine.

3.1 Deformation Framework

The Deformation Framework is designed as an isolated component that can be integrated into a game engine with ease. For achieving a reasonable performance, the framework is created with C++. Currently the method is structured so that different parts of it can be optimized in order to improve the high run-time requirements. The final framework will have the most critical parts of the algorithm parallelized, written with optimized instruction sets for the target platforms, and possibly partly ported to run on the GPU. The initial implementation was made to perform in an optimal way on a basic CPU to help conversion between different target platforms. In addition, the games tend to be more GPU than CPU bound due to requirements of visual perfection and special effects. There exists an implementation of quadratic elastic shape matching with documentation [2], which could serve as a reference if a GPU solution becomes more attractive in the future.

3.1.1 Data Structures

There are some containers and structural hierarchies that had to be defined. Figure 3.1 shows how the data is distributed between the components. From the main engine point of view, the model contains meshes. The meshes might contain batches if a further specialization is required but in a simple case, they contain only the vertex data. This is how the models are handled from the engine's point of view. On the other hand the shape matching requires its own structural definitions. Therefore, the deformable model consists of a cluster that is divided into cells. Each of the cells are connected to the actual vertex data.

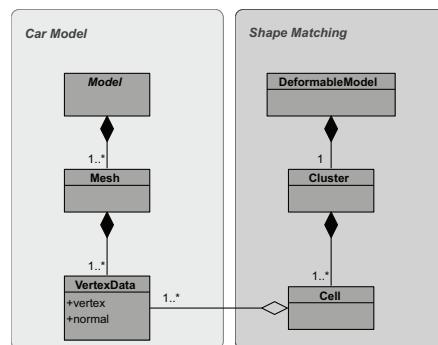


Figure 3.1. Data distribution in a deformation framework separates the shape matching part needed for deformation and the actual mesh data to be rendered.

3.1.2 General Operation Flow

The basic operation is driven outside the deformable model in order to make the framework as portable as possible to different kinds of system. In the case of an editor the actual scene handles the simulation in the following way, meaning that this set of operations is run continuously in this order while the simulation is running:

1. Physical simulation (applying gravity and other forces)
2. Collision detection and responses
3. Shape matching
4. Final integration to positions
5. Move vertices

The framework integrated into the game engine appears similar but contains a few exceptions in the ordering and that the loop is not run in a straight pipeline, but some steps are optional. The basic deformation for cars is done with a plastic material and thus running the simulation when no deformation occurs, meaning there are no collisions, would be done in vain. For elastic objects, however, the simulation would have to be run all the time.

1. Physical simulation is done
2. Collision detection and responses
 - (a) When a collision occurs the shape matching is started for plastic objects
 - (b) Final integration to positions
3. Another update thread updates the deformed vertices to be drawn if those have been changed

As shown above, the actual model mesh vertex positions are transformed separately. This can even be done in another thread and only when the vertices have been updated. This is possible because the result of the shape matching is updated to each cell's transformation matrix, which can then be used to render vertices in to their corresponding positions.

Pre-Calculation

As mentioned in the first chapter, the models in car games tend to consist of several separate parts or meshes. This makes modelling more clear as well as allowing the detaching of parts when they have reached a certain level of damage. For the shape matching it is not computationally reasonable to contain each of these parts as a separate deformation object but, instead, to store one model's information in a single deformation model. This requires some data restructuring.

Each vertex belongs to a certain mesh. Meshes, of course, have their normal and texture information. For the creation of deformable models, the mesh information has to be grouped into cluster cells. In this case, the vertices are assigned to cells. The simulation is run using the cells and then the information contained in each cell is updated according to the transformation of the cell. The information here being the vertices and normals for each mesh in a model. As shown before, the figure 3.1 describes structure of the data.

In his article, Müller [11] describes how clustering can improve physical plausibility of the simulation. Müller used overlapping cubical regions for the results in his paper, but also mentioned the possibility to use clustering with tetrahedrons. Actually, for the algorithm to function, almost any kind of clustering technique

can be used as long as the cells are overlapping. The overlapping regions are very important for holding the deformable object together. Chentanez [2] uses k-means clustering for his GPU-based implementation. For this thesis another method was also tested, where the data is divided in a k-tree sense, according to the amount of the data and considering the particles at the edges of the regions as the overlapping ones.

Even when the clustering may appear as insignificant a part of the shape matching implementation, it actually has an important effect in the final result. One of the problems with more dynamic sized cluster is the creation of the overlapping regions. For such cluster cells, a certain number of particles are chosen as representatives of the cell. Then the representatives should be connected to corresponding particles from the neighbouring cells. It is possible to select the particles at the edges of the cells, because that information can be retrieved from the shared triangles between the cells. Then these particles could be shared by the neighboring cells. In practice it seems however, that when the models get more complex and the particle masses increase, these relatively small overlapping bands are not sufficient to hold the model together anymore.

When considering the method used in the Müller’s paper with the cubical cells, the overlapping is simpler to perform, simply by using the neighbouring cells’ particles. In the cubical region, the usage of the cell particles is also more complete and thus can hold any structure together. Using tetrahedrons should work equally well, as the cells have the same amount of cornering particles that can be overlapped with the neighbours. The work presented in this thesis uses the cubical clustering approach, which is constructed when the model is loaded the first time.

The clustering in general has some problems that will arise. In a cluster of cells, each cell contains its transform matrices A_{qq} and A_{pq} as described in chapter 2.2 about shape matching. By using these matrices, the particles are moved to their optimal locations. The shape matching technique uses the least squares approach, and also a stiffness factor, α , to move particles into optimal positions. The possible problems arise when it is noted that each particle can be shared among multiple cells in a cluster. This results in multiple transforms and stiffness factors moving the same particles and their effect has to be combined. The approach used in the thesis was to take note of how many cells are using each particle, and then create a scaling factor: $scale = \frac{1}{n_{cells}}$, where n_{cells} is the number of cells using the particle. This scale is then used when calculating the goal positions in the shape matching simulation. This kind of averaging makes the particles match each other more accurately, however, similar matching cannot be done when applying the transformations to move the actual vertex data. This will in some cases result in visually unpleasant effects with overlapping vertices.

Collision response

Generally the physics of the game engine, or in some cases a separate physics engine, handles the collisions in phases. It is usually assumed that the geometry is divided into so-called *islands*, which contain possibly colliding objects. These islands are checked for possible collisions in a so-called *Broad Phase*. The possibly colliding pairs are then sent for the *Narrow Phase* testing, which generates collision points with collision normals and momenta. In the broad phase collisions are checked mainly by using some collision volumes such as axis-aligned bounding-boxes (AABB), bounding spheres or oriented bounding-boxes (OBB). The narrow phase can continue checking bounding volume against polygons, or to perform more intensive polygon-to-polygon checks.

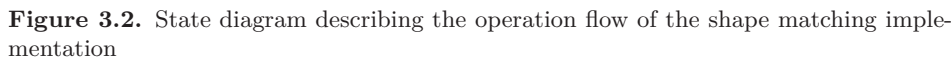
As the implementation is done in the game engine the deformation model receives information from the physics library about the contact with a certain number of contact points, the contact normal and a momentum of the collision. In the shape matching simulation, this collision event is used to move the cluster's cell particles to their target position. From the point of view of the algorithm this generates the set of x_i , and is used with the x_i^0 to generate the goal positions with the shape matching algorithm.

Shape Matching

Figure 3.2 describes the whole operation flow of the implemented shape matching framework. In the pre-process stage the mesh is loaded and the cell cluster is constructed. In the next stage the static centres of masses are calculated for all the cells. The symmetric scaling matrices A_{qq} are calculated, using the static particles x_i^0 . Note that an A_{qq} matrix contains no rotation, only the scaling. Vertices need to store and, in plastic cases update, the static transformation matrix describing the state where the x_i^0 particles are currently positioned. Vertices are moved at each time step. This stage can also be done in a separate update thread since the vertices to be drawn are given by moving the original vertices with transformation matrices, not by iterating their positions, and so it does not matter if some iterations are not done, or are done two times during the simulation.

The pre-integration step calculates the intermediate position and velocity using Euler integration (see equations 2.1 and 2.2). Müller[11] states that, the shape matching improves the stability of the Euler integration by using the centres of masses for the actual transition. For a pre-integration step Verlet integration could still be used to improve the input, x_i , to shape matching (see equation 2.3).

The actual real-time shape matching simulation begins by calculating the current centres of masses for all the particles. After this information is gathered the A_{pq} can be obtained. It is already possible to see if flipping has occurred by testing



In most cases flipping does not occur and the optimal transformation can be calculated. Polar decomposition [18] can then be used to separate the rotational (R) and scale/stretch(S) parts. For this state, it has to be noted that Chentanez mentions possible problems with singular matrices [2]. This may be due to the fact that his implementation relied on using quadratic shape matching since in the test environment, singular matrices were not a common problem in this phase.

Conservation of volume is as simple as dividing the matrix A by the cubic root of its determinant. This was left as configuration option as in some dominantly linear transformation cases it tends to make the system unstable. After the previous stages, the final transformation matrix T can be formed by combining the fully rotational matrix with the optimal one by using a blending factor, β . This factor states the amount of the linear part taken into the final transformation. For the

system to function properly β should have values from the range $[0, 1)$. When the β is close to 1, the system cannot hold the object in shape since there is no real support.

Using the previously obtained transformation matrix, the target positions can be calculated. In the cluster-based test environment, more than one iteration through the shape matching part was required to then obtain reasonable stiffness with more dense cell clusters. If more than one iteration is used then the x_i are replaced by g_i and the shape matching is run again.

The next step is to move the particles to their final locations using the equations 2.21 and 2.22. The plasticity yield is checked in order to find out if the static matrices A_{qq} should be updated. Müller's paper[11] proposes the calculation of a so-called *plasticity yield* by taking a *spectral norm* of the (S-I). However, the calculation of spectral root requires identifying the largest singular value of a matrix, which would require an eigenvalue decomposition. This is not computationally feasible and, when consulted, Müller mentioned that what was actually used in his system was the Frobenius norm. There is just a misunderstanding in the notation of the original paper. Frobenius norm in equation 3.1 is calculated very simply compared with the spectral norm.

$$\|A\|_F^2 = \sum_{i=1}^m \sum_{j=1}^n |a_{ij}|^2 \quad (3.1)$$

The resulting value of the plasticity yield is compared with a user-specified yield value. If the calculated yield exceeds, plastic deformation occurs, and the static matrices are re-calculated, otherwise the simulation continues by applying more forces or waiting for collision responses from the system.

3.2 Additional stiffness

Stiffness or density of the material is in Müller's paper[11] adjusted with a parameter α , as shown in the equation 2.21. This works well for objects contained in a single cluster, however, as described above, in order to simulate material properties, such as bending of metal, using linear transforms, the number of needed cluster cells increases. The number of cluster cells determines the flexibility in local deformations. The increased cluster size, however, brings some problems. In addition to increased computational cost due to the shape matching, the stiffness specified by the α loses its effect because of the averaging with the neighbors. The simple stiffness factor has an effect inside a single cell, nevertheless, when the amount of clusters is doubled, the amount of stiffness due to the α is approximately halved as there are more cells each having their neighbours affecting them. See figure 3.6 for the effect.

The problem could be solved in a number of ways. The traditional approach is to iterate inside the shape matching loop for a certain amount of time. Some sort of priority iteration according to the collision position could be added as well. Another way would be to have a sort of hierarchical cluster. The effect of the stiffness factor, α , has more effect on a small amount of cells, meaning that the larger cells in the hierarchy could be used for increased stiffness. But when large enough impact would occur, the smaller cells would provide more detailed deformation.

The method used in this implementation was the first one, the basic iteration, as it did not require the reconstruction of the whole clustering system. The other solution might be a reasonable subject for further study, as it would also allow the application of other parameters in a hierarchical manner. Adding the priority for the iteration is considered only a minor optimization.

3.3 Deformation Editor

Most of the development of the deformation framework was done outside the main game engine. This was to allow an isolated testing environment and to ensure that the deformation framework could function as a separate component. For the purpose of running multiple tests with various setups and to allow real time modification of parameters, an editor named "DeformEd" was created. The editor allows dynamic object loading, creating clusters of various sizes and, most importantly, the deformation of models in real-time. The figure 3.3 shows the editor interface. The actual colours are changed for printing purposes.

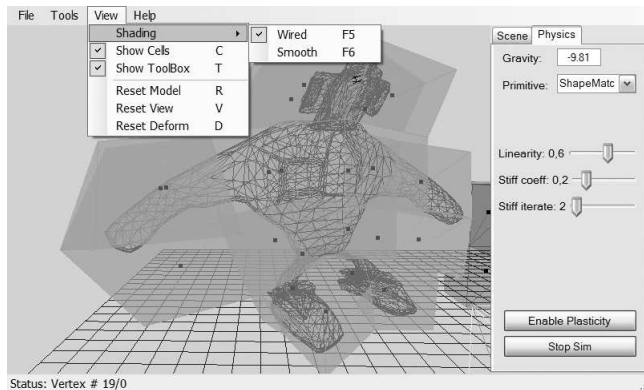


Figure 3.3. Editor interface to the deformation framework, showing a model inside a very simple cluster of cells, and the basic parameters for controlling the deformation.

The editor and its graphical user interface (GUI) was created with C# programming language, this further encourages the component model of the deformation framework. The deformation framework is loaded by the editor as a Dynamic Link Library (DLL). This division is shown in the class diagram, figure 3.4. While the creation of the external editor was clearly a benefit for the simulation properties in its isolated environment, the implementation was not as simple, because many of the features already present in the engine could not be just extracted and used for the editor. For this reason, some of the functionality was built from the ground up, such as were object file loading, a simple DirectX rendering framework with some debug visualization capabilities, mouse interaction for selection, interaction and camera control, simple physics, collision response, animation components, and dynamic parameter adjustment from the GUI.

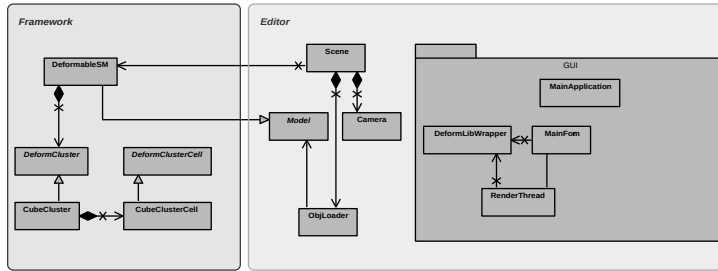


Figure 3.4. Class diagram: the editor and the deformation library.

With the editor many of the features are easily visualized. Figure 3.5 shows the difference between mostly linear and mostly rotational transformations, i.e. the variation of the β parameter in equation 2.19. The vertex on the floor is fixed at its original position, and the upper one was pulled to the right. The algorithm allows blending between the two extremes to achieve varying behaviour between the rigid and more elastic materials.

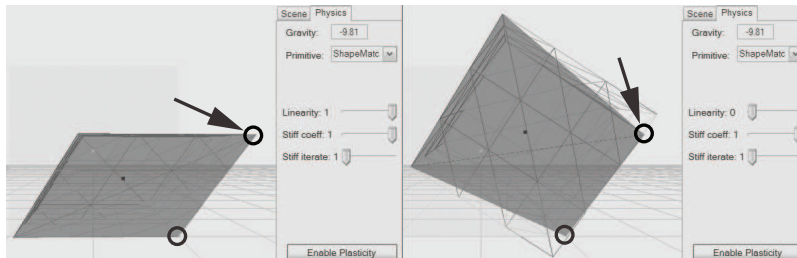


Figure 3.5. Linear- and rotational transform using the same fixed vertex positions.

Figure 3.6 gives the impression how the cluster size has an effect on the flexibility of material. Not only that but when the cluster size increases the stiffness decreases. The images in the figure all have the same material properties, only the cluster size is varied. This visualization demonstrates the need for the additional stiffness factor in addition to the one presented in the reference paper[11], as described in chapter 3.2.

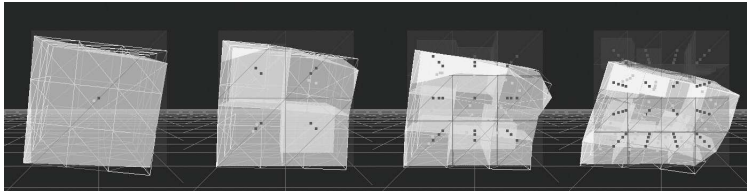


Figure 3.6. Varying cluster sizes (1, 8, 27, 64), shows how the increased amount of cells decreases the stiffness of the object.

The body of an actual in-game car model under manually adjusted deformation inside the editor is shown in the figure 3.7. The cluster size here is 39 and appears adequate for a basic level of deformation. The future development for the editor might include manually adjustable clustering to provide more detail where it is required.

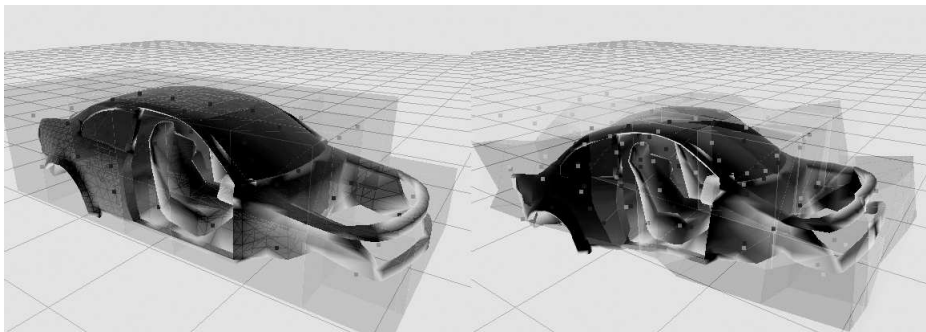


Figure 3.7. Original and deformed car in the editor

3.4 Integration with the Game Engine

The integration with the game engine was required to obtain the actual benefit of the deformation framework. The game engine is the core of every game, providing the necessary tools to create the actual game code. Some companies purchase the engine from a third party while others use custom built solutions. Game developing companies may also purchase middleware solutions for specific areas of their choice, such as physics, menu-, or tree generation. This is a growing trend as the developers do not always have the required resources to create their own specific solution, or have an agreement with the publisher to use something of their specification. There are clear benefits if the company's own game engine code consists of isolated components since the replacement of any individual part can be a simple task of synchronizing interfaces.

As the deformation framework was designed from the start as an isolated component, the replacement of the old deformation system in the game engine was relatively straightforward. Still the integration required the following things to be added and changed in the framework:

- The actual car game model to be deformed can consist of several meshes with their respective transforms, instead of just one in the editor. (for the deformation to be consistent over a model the meshes were mapped to a single deformation model)
- Not only the vertices of the meshes had to be transformed but the normals, bi-normals and tangents as well.
- Dynamic adjustment of parameters appropriate for different materials.
- Support for additional components inside a cluster that can have parameters of their own.
- Automating the setting of common parameters for specific car parts (engine, base, etc).
- Many visual tools to improve the visual debugging.
- Platform specific optimization.

Whereas the isolated testing environment for the purposes of developing the deformation framework were a clear benefit, the creation of similar testing facilities inside a running game environment are beneficial when fine-tuning the adjustable parameters of the deformation system. Similar visualization is provided, as well as a run-time parameter adjustment through the external interface existing in the game engine. The figure 3.8 shows a visualization of a cluster after some car deformation in the test environment.

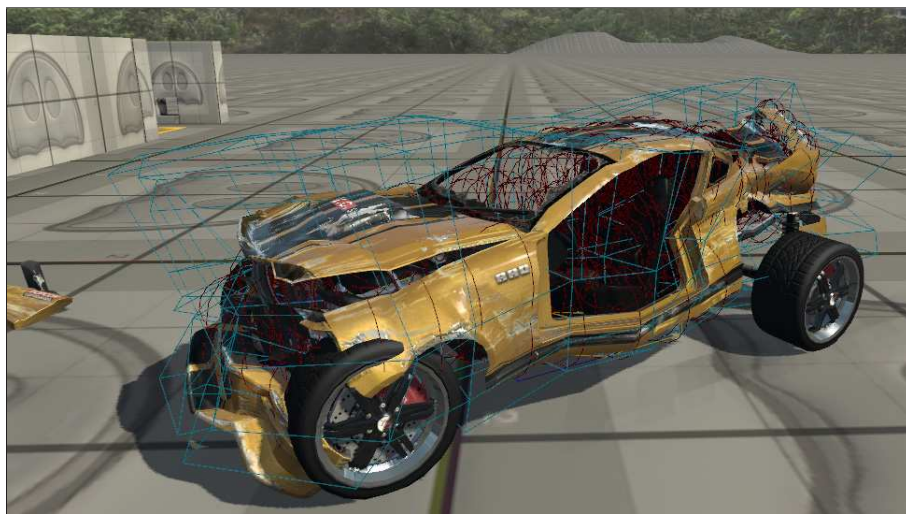


Figure 3.8. shape matching deformation in test environment.

The ongoing optimization study and comparison with the game engine’s previous deformation system will determine to what degree the shape matching system will be taken up for use. The performance of the simulation is very good even running on CPU, but another consideration is whether the system can also be used for deforming objects other than cars. The visual comparison between the artist-modelled deformation, and the software-generated shape matched one, seems that both are offering something that the other cannot achieve. This could mean that a combination of these two would give the best visual results. In this kind of a hybrid approach the shape matching would be used for larger scale deformation, whereas the artist-modelled technique would provide high frequency details. The final result would be a blend between these two systems.

3.5 Plasticity extension

The primary purpose of the research was to simulate objects with plastic features. Plasticity was briefly discussed in Müller’s paper[11], but the specific details about finding the correct plasticity coefficients and its placement in the deformation pipeline were not outlined. The paper suggests using a norm of the linear part of the matrix obtained with the least squares method, see 3.1.2 for more detailed description. If the norm exceeds the user specified *plasticity yield*, deformation

occurs. The paper also mentions two other parameters for controlling the plasticity, *creep* and *maximum plasticity*. The creep determines the slope or the speed of the deformation, and the maximum plasticity determines how much deformation can occur during one time step. However, when experimenting with a system having only linear deformations for the plasticity, and especially if using volume-conserving cells, the system quickly becomes unstable when the deformations get too extreme. The rotational part seems to make the system more robust in the sense that the system does not over-shoot. This raised the idea to experiment whether the rotational part of the deformation could be included into the plasticity as well.

The changes required for the inclusion of the rotational component are not difficult to implement. Instead of calculating the norm of $(S - I)$, using the linear part from equation 2.18, the norm is calculated using the blended matrix containing the both parts $(T - I)$, the T is calculated with the equation 2.19. When the norm exceeds the yield, the control particle positions are updated by the transform. The transformations, U , that are used to dynamically transform the original set of mesh vertices are updated for the cells as follows: $U_{vertices} = TU_{vertices}$. Finally the static centres of mass need to be moved to the current centres of masses, i.e. $x_{cm}^0 = x_{cm}$ for all the cells. This is because the transformations are given related to those positions and cannot be re-calculated without unnecessary overhead. It has to be noted that a separate transformation between the static particle positions and the original ones is not stored as it is more efficient to apply the static part directly to the control particles. The following figures, 3.9 and 3.10, compare the transformation with only linear plasticity and another with the rotation included.

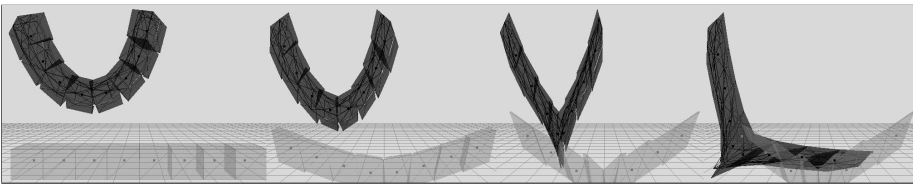


Figure 3.9. Original simulation state, two steps of linear plasticity animation, and release. Here no bending is achieved.

In the first figure 3.9 it can be seen how the linear plasticity cannot capture any kind of bending deformation. In meshes that are more complex this also introduces some inabilities when the neighbouring cells cannot agree on matching linear deformations, and keep on pushing each other further away.

Figure 3.10, shows the plasticity with the full simulation transform included. The

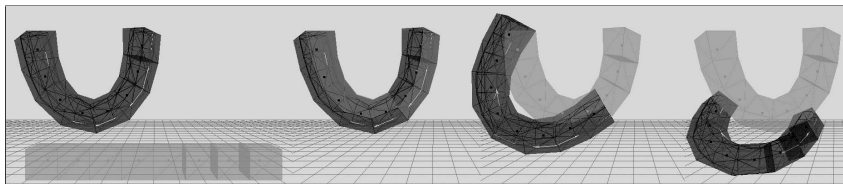


Figure 3.10. Original simulation state, one step of plasticity, release and fall. Here the object has been bent.

current implementation does not perform the creeping that would blend the transform. This is because the metallic deformation, that occurs in car crashes, is more instantaneous and after the plasticity yield is reached the deformation stays.

3.6 Transforming normals

One thing to take note of is the re-calculation of the lighting normals of the original vertices. The same transforms that are used for transforming the vertices can be used, but one has to remember that when normals are transformed it has to be done as shown in the equation 3.2 below, by using the transpose of the inverse of the original transformation. For the formula derivation see Matt Pharr's online article[14].

$$n_{new} = (T^{-1})^t n_{old} \quad (3.2)$$

When one is using other vertex properties similar to normals, such as bi-normals and tangents those can be updated in the same way.

Chapter 4

Conclusions

4.1 Results

Achieving realistic simulation of object deformation in real time is a goal towards which research will continue to strive. There are many approaches to the problem, but some are more suitable for the specific simulation needs and some for others. A general-purpose solution with a full physical accuracy might not even be desired due to the additional requirements and workload. Thus, especially in the game industry, solutions approximating a visually pleasing accuracy will continue to dominate.

The *Meshless deformations based on shape matching* method [11] proved to successfully give real time capability to simulate plastic deformations. The method had to be extended at some points to increase the stiffness of objects with a high number of cells in a deformation cluster. Another important addition was the inclusion of the rotational term in the plastic part, which allowed support for discrete bending using only the linear least squares approach. The meshless or point-based methods seem prominent due to their reasonably simple tuning and flexibility. To further improve the visual appearance by adding fine detail, the framework might use a form of hybrid implementation with the original mesh morphing method. The method is also suitable for GPU implementation, this is however, more of a workload balancing issue, as the GPU usage may not be the better option due to already heavy load on GPU shaders.

All in all, the results were good with the selected method for the implementation. The method proved to be simple enough, yet providing the flexibility to support several materials and other features.

This thesis provided an overview about the most used methods for mesh deformation and gave basic mathematical background to start looking into these as well.

4.2 Future Work

The deformation framework will continue as a part of the game engine, and will probably have some other presented methods included. This would give the possibility for an extensive comparison in a real time environment. To further increase the value of the deformation system, it should support fractures. This would require fast mesh capping techniques and probably some additional adjustable parameters for the cells of the cluster.

Overall, the work around the damage system's deformation and fracture techniques is something of interest for everyone doing real time simulations, as the current technology makes it possible to perform things like never before. The possibilities of simulating physical phenomena more accurately and on a larger scale, increase when the technology evolves further. Recently there has been a lot of interest in finite element implementations and it seems to be the way that research is going to proceed in. As in many research topics in the computer graphics and physics simulation, the possibilities improve when increased preprocessing capabilities and increased customer hardware performance become available.

4.3 Summary

This thesis has covered an introduction to deformation in an interactive computer simulation. The discussion here had the focus on generating real time deformation simulation for a car racing game environment. The required theory behind and methods that are currently in use were described as well as the basic mathematical background into these.

For the described purpose of improving dynamic collisions in a car racing game, a method called *Meshless Deformations Based on Shape Matching* was described. The main parts being the shape matching, dividing the mesh data into cluster of cells to add flexibility in deformation, and the implementation of plasticity.

The implementation chapter described the structure of the deformation framework. In addition, issues specific to the implementation are discussed to indicate possible obstacles as well as possibilities to improve upon. Two extensions to the original method were given: *additional stiffness* and inclusion of the *rotation in the plasticity*. The implementation was described with diagrams and its capabilities presented with screen captures to provide a better idea of the functionality of the framework and the theory.

Bibliography

- [1] Alan H. Barr. Global and local deformations of solid primitives. *SIGGRAPH Comput. Graph.*, 18(3):21–30, 1984.
- [2] Nuttapong Chentanez. Deformable body simulation on GPU. *Nvidia Developer Forum*, 2007. <http://developer.download.nvidia.com/whitepapers/2007/SDK10/DeformableBody.pdf>.
- [3] Halliday David, Resnick Robert, and Walker Jearl. *Fundamentals of Physics Extended*, chapter 10. John Wiley & Sons, Inc, fifth edition, 1997. ISBN 0-471-10559-7.
- [4] Furio Ercolessi. A molecular dynamics primer: The verlet algorithm. Website, October 1997. <http://www.fisica.uniud.it/~ercolessi/md/md/node21.html>.
- [5] Norbert Frisch and Thomas Ertl. Deformation of finite element meshes using directly manipulated free-form deformation. In *SMA '02: Proceedings of the seventh ACM symposium on Solid modeling and applications*, pages 249–256, New York, NY, USA, 2002. ACM.
- [6] Alex Henriques and Burkhard Wünsche. Improved meshless deformation techniques for real-time interactive collaborative environments. In *GRAPP (AS/IE)*, pages 154–164, 2007.
- [7] Robert Hooke. *Lectures de Potentia Restitutiva*. John Martyn, London, 1678.
- [8] Thomas Jacobsen. Advanced Character Physics. *Game Developers Conference*, 2001.
- [9] Steve McMillan. The Leapfrog Integrator. Website, May 2001. http://www.physics.drexel.edu/courses/Comp_Phys/Integrators/leapfrog/.
- [10] Turner MJ, Clough RW, Martin HC, and Topp LJ. Stiffness and deflection analysis of complex structures. *Journal of the Aeronautical Sciences*, 23:805–823, 1956.
- [11] Matthias Müller, Bruno Heidelberger, Matthias Teschner, and Markus Gross. Meshless deformations based on shape matching. In *SIGGRAPH 2005: ACM*

- SIGGRAPH 2005 Papers*, pages 471–478, New York, NY, USA, 2005. ACM Press.
- [12] Patricia Moore and Derek Molloy. A survey of computer-based deformable models. In *Machine Vision and Image Processing Conference, 2007. IMVIP 2007. International*, pages 55–66, 2007.
 - [13] Andrew Nealen, Matthias Muller, Richard Keiser, Eddy Boxerman, and Mark Carlson. Physically Based Deformable Models in Computer Graphics. *Computer Graphics Forum*, 25(4):809–836, 2006.
 - [14] Matt Pharr. Transforming normals. Website, August 2005. http://www.cgafaq.info/wiki/Transforming_Normals.
 - [15] Alec R. Rivers and Doug L. James. FastLSM: fast lattice shape matching for robust real-time deformation. *ACM Trans. Graph.*, 26(3):82, 2007.
 - [16] A. Samuelsson and O.C. Zienkiewicz. History of the Stiffness method. *International Journal for Numerical Methods in Engineering*, 67:149–157, 2006.
 - [17] Thomas W. Sederberg and Scott R. Parry. Free-form deformation of solid geometric models. *SIGGRAPH Comput. Graph.*, 20(4):151–160, 1986.
 - [18] Ken Shoemake and Tom Duff. Matrix animation and polar decomposition. *Proceedings of Graphics Interface '92*, pages 258–264, 1992.
 - [19] undefined. *The American Heritage Dictionary of the English Language*. Houghton Mifflin Company, fourth edition, 2004. ISBN 0-440-23701-7.
 - [20] Loup Verlet. Computer "experiments" on classical fluids. i. thermodynamical properties of lennard-jones molecules. *Phys. Rev.*, 159(1):98, Jul 1967.

Appendix A

Deformation Methods in Use

The following table is a brief summary of the article in Computer Graphics Forum 2006 [13]. The paper describes the current methods in use in the field of Physically Based Model Deformation briefly, and presents their potential drawbacks as well as gives references to relevant material. The table below was created to be used together with the article to more quickly find out the method with desired properties and some of the most important references. For more in depth details reader is advised to refer the original papers in question.

Lagrangian Mesh Based

Finite Element(FEM)

- solve PDEs on irregular grid
- computations on the volume
- explicit, implicit (computationally heavy) and semi-implicit
- some real time implementations

Finite Differences(FD)

- semi-implicit
- boundary issues
- local adaptations only for irregular meshes
- extended for plasticity and fracture

Finite Volume

- improvement over FEM

Boundary Element (BEM)

- computations on the surface
- interior is assumed homogeneous
- integral of motion from 3d to 2d
- topology changes difficult
- achieves real time performance

Mass-Spring Systems

- Point-Mass network (construct with discrete model)
- solve system of coupled ODEs
- shear springs, structural springs
- viscosity force with damping
- many real time implementations
- problems with stability

Particle Systems

- extended Mass-Spring with deformation energies
- forces are used to preserve distances, areas, volumes
- separate models for bending

Lagrangian Mesh-Free

Loosely Coupled Particle Systems

- objects are without well-defined surface
- spatially coupled interaction (speed up when restricted to neighbours)
- can have implicit surface representation
- problem with volume conservation
- influence graph for a sphere of influence
- simulation of granular materials

Smoothed Particle Hydrodynamics

- discrete, smoothed particles approximate values
- mass is trivially conserved
- suited for RT-applications
- not possible to maintain incompressibility
- implicit coating
- Used for many things: fluids, hair, deforming bodies, etc.

Mesh-Free

Point based animations

- mesh-free physics and point sampled surfaces
- first-order accurate (moving least squares)
- Simulates stiff elastic to highly plastic materials
- expendable to fluids
- RT-performance possible
- implicit for fluids, explicit for solids
- extended to support fractures
- deformable collision detection
- surface skinning
- can be efficiently raytraced